TypeScript 3~4

タイプ

Typescriptには値を推論する方法が二つある。

union : 値に可能なタイプを二つ以上にすること。

narrowing : 値に可能なタイプが二つ以上にならないようにすること。

```
let a: string | undefined
let a = Math.random() > 0.5
    ? undefined
    : "Yun gi hak";
```

unionはこのように色々なタイプができることである。(string, undefined)

```
let a :number | string = 34;
a.toString();
a.toUpperCase();
```

そして、unionは可能なタイプが色々あるので、すべてのタイプが可能なメソッドだけしか使えない。

このようにnumberとstringが可能なtoString()はできるけど、stringだけできるtoUpperCaseはできない。

```
let a :number | string;
a = 123;

a.toUpperCase();
```

narrowingはこのように複数のタイプをa=123;をしてnumberに制限する。

```
let a :number | string;
a = 123;

a = "yungihak";
a.toUpperCase();
```

でも、このようにstringをaに割り当てればstringに変わる。

```
const a: "yungihak"
const a = "yungihak";
```

リテラルタイプ一般的なstring, numberとは違う。

stringは"abc", "a"などの色々な値ができる。しかしリテラルタイプはタイプを設定して、唯一の値になる。

タイプエイリアス

```
let a: boolean | number | string | null | undefined;
let b: boolean | number | string | null | undefined;
let c: boolean | number | string | null | undefined;
let d: boolean | number | string | null | undefined;
let e: boolean | number | string | null | undefined;

type alltype = boolean | number | string | null | undefined;
```

Typescriptはこのようにタイプに名前を付けて使うことができる。

オブジェクトタイプ

v6.0.3 Run Export Share

```
const me = {
    name: "yungihak",
    age: 29
}

me['name'];
me['age'];

console.log(me['name']);
console.log(me['age']);
```

```
[LOG]: "yungihak"
[LOG]: 29
```

このようにオブジェクトをつくることができる。

そして、value['member']の形で使う。

v6.0.3 Run Export Share

```
let me: {
    name: string;
    age: number;
};

me = {
    name: "yungihak",
    age: 29
}

console.log(me['name']);
console.log(me['age']);
```

```
[LOG]: "yungihak"
[LOG]: 29
```

一般的にはこのようにタイプを表示する。

オブジェクトタイプUNION

```
const man = Math.random() > 0.5
    ? { name: "yunghiak", age: 29}
    : { name: "honggildong", live: false};


man.name;
man.age;
man.live;
```

このようにオブジェクトタイプにUNIONを使うことができる。

```
man.name;   (property) name: string
```

nameはstring

```
man.age;   (property) age?: number | undefined
```

ageは　number | undefined

```
man.live;   (property) live?: boolean | undefined
```

liveは　boolean | undefinedになる。

もちろんタイプを表示することもできる。

v6.0.3 Run Export Share .JS .D.TS Errors Logs Plugins

```typescript
type PoemWithPages = {
    name: string;
    pages: number;
    type: 'pages';
};


type PoemWithRhymes = {
    name: string;
    rhymes: boolean;
    type: 'rhymes';
};

type Poem = PoemWithPages | PoemWithRhymes;

const poem: Poem = Math.random() > 0.5
    ? { name: "A", pages: 6, type: "pages"}
    : { name: "B", rhymes:true, type: "rhymes"


if (poem.type === "pages") {
    console.log(`poem pages :  ${poem.pages}`)
} else {
    console.log(`rhymes : ${poem.rhymes}`);
}

poem.type;
```

```
[LOG]: "poem pages :  6"
```

これは判別されたUNIONである。

オブジェクトの中でタイプを作る。

そして、narrowingを使ってタイプを決定することにする。

```
(property) type: "pages" | "rhymes"
poem.type;
```

Logにはタイプをpagesに表示されても
タイプは　"pages | rhymes"である。